TASKORA

Team Management Application

SOFTWARE REQUIREMENTS DOCUMENT

Version 1.0 | 2025

Prepared by:

Raynald (Programmer) | Duard (System Analyst)
Yovela (Product Owner) | Nicholas (Tech Lead)

CONFIDENTIAL

1. Introduction

1.1 Purpose of this Document

This Software Requirements Document (SRD) defines the complete functional and non-functional requirements for Taskora, a web-based team management application. It serves as the authoritative technical reference for all development, design, and deployment activities, and is intended for use by developers, system analysts, product owners, stakeholders, and academic evaluators.

1.2 Project Overview

Taskora is a collaborative task management platform designed to help teams organize their work, assign responsibilities, track individual contributions, and meet deadlines efficiently. The application targets student groups, startup teams, and small-to-medium professional teams that suffer from coordination overhead and lack of accountability in distributed workflows.

1.3 Project Scope

Taskora covers the following scope:

- User authentication and role-based access control
- Project and team workspace management
- Task creation, assignment, and lifecycle management
- Contribution tracking per team member
- Calendar integration and deadline reminders
- File/resource shortcut sharing within workspaces
- Task commenting and verification workflows
- Optional team discussion forum
- RESTful API backend with PostgreSQL persistence

1.4 Intended Audience

Audience	Usage of Document
Development Team	System design, coding standards, API specification
Product Owner	Feature prioritization, requirement validation
Tech Lead	Architecture review, technical decisions
Academic Evaluators	Assessment of technical completeness and engineering rigor
Future Stakeholders	Understanding system capabilities and constraints

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
SRD	Software Requirements Document
API	Application Programming Interface
REST	Representational State Transfer — architectural style for web APIs
JWT	JSON Web Token — standard for secure stateless authentication
DTO	Data Transfer Object — validated input/output schemas
RBAC	Role-Based Access Control
ORM	Object-Relational Mapper — abstracts database queries
CI/CD	Continuous Integration / Continuous Deployment
UX	User Experience
UI	User Interface

2. Overall Description

2.1 Product Perspective

Taskora is a standalone web application that operates as a full-stack system with a React frontend, a TypeScript/Node.js backend, and a PostgreSQL database. It integrates no third-party task management platforms but may integrate third-party calendar services (e.g., Google Calendar) and email notification providers in future iterations. The system is entirely self-contained for its core feature set.

2.2 Problem Statement

Modern collaborative teams — especially in academic and early-stage startup environments — face systemic coordination failures. These failures are not attributable to individual incompetence but arise from structural gaps in tooling and process. The primary problems identified are:

- Unclear task distribution: Without a unified task board, responsibilities blur and overlap.
- Missed deadlines: No centralized deadline tracking leads to forgotten commitments.
- Uneven workload: Without visibility into individual contributions, workload imbalance goes uncorrected.
- No objective contribution measurement: Contribution assessments rely on subjective perception rather than data.
- Resource access difficulty: Files and links are scattered across multiple platforms.
- Limited project visibility: Team leaders lack a real-time overview of project progress.

2.3 Product Functions Summary

At a high level, Taskora provides the following capabilities:

- Secure authentication and role-based authorization
- Team and project workspace management
- Task lifecycle management (create, assign, update status, verify, close)
- Quantitative contribution tracking per member per project
- Calendar view with integrated deadlines
- Automated deadline reminder notifications
- Shared folder shortcuts for centralized resource access
- Task-level commenting for asynchronous discussion
- Task verification by authorized team members
- Optional discussion forum per team workspace

2.4 User Classes and Characteristics

Role	Permissions	Characteristics
Admin	Full system access, user management	System-wide administrator, manages all workspaces
Team Leader	Create/manage projects, assign tasks, verify tasks, add/remove members	Responsible for team coordination and oversight
Team Member	View/update assigned tasks, add comments, access resources	Executes assigned work, collaborates with peers
Viewer	Read-only access to project data	Stakeholders or observers with limited access

2.5 Operating Environment

- Frontend: Modern web browsers (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)
- Backend: Node.js 18+ runtime environment
- Database: PostgreSQL 14+
- Deployment: Cloud VPS or container-based hosting (Docker)
- Network: HTTPS with TLS 1.2+

2.6 Design and Implementation Constraints

- The backend must be implemented using TypeScript with strict mode enabled.
- All API endpoints must follow RESTful conventions with proper HTTP semantics.
- JWT tokens must use RS256 or HS256 signing with configurable expiration.
- All user passwords must be hashed using bcrypt with a minimum cost factor of 10.
- Database schema must use proper normalization, foreign keys, and indexing.
- The system must support concurrent users without race conditions in contribution tracking.

3. Main Requirements & Priority Analysis

3.1 Main Requirements Identification

The main requirements of Taskora are derived directly from the core problems identified in Section 2.2. They are the non-negotiable capabilities that the system must deliver to be considered functionally complete and valuable to its users.

3.1.1 Authentication & Authorization (CRITICAL)

The system must authenticate all users via JWT-based login and enforce role-based access control throughout all operations. Without this, data integrity, user privacy, and contribution accountability are impossible. This requirement underlies every other feature.

3.1.2 Task Management (CRITICAL)

Users must be able to create tasks with titles, descriptions, deadlines, priorities, and statuses. Tasks must be assignable to specific members. Status transitions (To-Do, In Progress, Under Review, Completed) must be tracked and auditable. This is the central workflow of the system.

3.1.3 Contribution Tracking (HIGH)

The system must automatically calculate and display each member's contribution score based on task completion, verified tasks, and timeliness. This addresses the most persistent pain point: lack of objective participation measurement. Contribution data is updated transactionally whenever a task status changes.

3.1.4 Deadline Management (HIGH)

Calendar integration and automated reminder notifications are required to prevent missed deadlines. Tasks with approaching or overdue deadlines must be surfaced prominently in the UI and trigger backend notification jobs.

3.1.5 Team & Workspace Management (HIGH)

Team leaders must be able to create workspaces, invite members via email or link, define roles, and remove members. Workspaces serve as the organizational container for all projects and their associated tasks.

3.1.6 Resource Sharing (MEDIUM)

Shared folder shortcuts allow teams to consolidate links to external resources (Google Drive, Notion, etc.) within the workspace. This eliminates the need to hunt for files across multiple platforms.

3.1.7 Task Comments & Verification (MEDIUM)

Task-level commenting supports asynchronous communication. Task verification by a Team Leader or peer provides a formal quality gate before tasks are marked complete.

3.2 Priority Matrix

Requirement	Priority	MVP (v1.0)	Rationale
Authentication & Authorization	CRITICAL	Yes	Foundation for all other features
Task Management	CRITICAL	Yes	Core value proposition
Contribution Tracking	HIGH	Yes	Primary differentiator from competitors
Deadline Management	HIGH	Yes	Directly solves missed deadline problem
Team & Workspace Mgmt	HIGH	Yes	Required for multi-user operation
Resource Sharing	MEDIUM	Yes	Reduces context switching
Task Comments	MEDIUM	Yes	Supports async collaboration
Task Verification	MEDIUM	Yes	Quality gate for contribution scoring
Discussion Forum	LOW	No	Optional enhancement for v2.0

3.3 Integration of Requirements into the Proposal

The requirements above are integrated into Taskora's design through four architectural decisions:

1. Authentication and RBAC are enforced at the Router and Middleware layers, ensuring every request is validated before reaching business logic.
2. Task management and contribution tracking are handled in the Service layer with transaction management to ensure data consistency.
3. Deadline management is implemented as a background job scheduler (cron-based) that queries overdue tasks and dispatches notification events.
4. Resource sharing and comments are first-class entities in the database schema, enabling relational queries and future extensibility.

4. Technology Stack Justification

4.1 Frontend: ReactJS + Vite

ReactJS is the industry-standard library for building component-based user interfaces. Its declarative programming model simplifies complex state management, while its large ecosystem provides ready-made solutions for routing (React Router), state management (Zustand/Redux), and UI components (Shadcn/ui, Tailwind CSS).

Vite is chosen as the build toolchain over Create React App (CRA) for several compelling reasons:

- Development speed: Vite uses native ES modules, resulting in near-instant hot module replacement (HMR) compared to CRA's Webpack-based cold starts.
- Build performance: Vite's Rollup-based production builds are significantly faster and produce smaller bundles.
- Modern defaults: Built-in TypeScript support, CSS modules, and tree-shaking without configuration overhead.
- Active maintenance: Vite is actively maintained by the Vue and Vite communities, with strong adoption in the React ecosystem.

4.2 Backend: TypeScript on Node.js

TypeScript extends JavaScript with a rich type system that catches errors at compile time rather than runtime. For a production backend this is invaluable:

- Type safety: DTOs, service interfaces, and repository contracts are statically typed, preventing null reference errors and incorrect data shapes.
- Code maintainability: IDE-level autocompletion and refactoring support accelerate development and reduce cognitive load.
- Layered architecture compatibility: TypeScript interfaces map cleanly to the defined boundaries between layers (Router, Controller, Service, Repository).
- Ecosystem maturity: Libraries like Express, Fastify, Prisma, and TypeORM have first-class TypeScript support.
- Team knowledge: TypeScript allows JavaScript developers to adopt types incrementally without rewriting existing code.

4.3 Database: PostgreSQL

PostgreSQL is the most capable open-source relational database available and is the correct choice for Taskora given its relational data model:

- ACID compliance: Full transaction support ensures contribution scores are never corrupted by concurrent updates.
- Relational modeling: Foreign keys, join tables, and constraints enforce data integrity between users, teams, tasks, and contributions.

- Indexing: B-tree and partial indexes improve query performance on task lists filtered by status, deadline, or assignee.
- JSON support: PostgreSQL's JSONB column type provides flexibility for storing semi-structured metadata without sacrificing relational integrity.
- Scalability: PostgreSQL handles millions of rows efficiently with proper schema design, and supports read replicas for horizontal scaling.
- Cloud availability: Available as a managed service on AWS RDS, Google Cloud SQL, Supabase, Railway, and Render.

4.4 Backend Architecture: Layered Architecture

A layered (N-tier) architecture is chosen over microservices or monolithic unstructured code for the following reasons:

- Appropriate complexity: Taskora's initial scale does not justify microservices overhead. Layered architecture provides structure without premature optimization.
- Separation of concerns: Each layer has a single responsibility, making the codebase easier to navigate, test, and modify.
- Testability: Service and Repository layers can be unit tested in isolation with mocked dependencies.
- Team onboarding: New developers can understand the architecture by reading the layer definitions without needing to trace distributed service calls.
- Incremental scalability: Individual layers can be extracted into microservices if load demands it in future versions.

5. System Architecture Design

5.1 High-Level Architecture

Taskora follows a classic client-server architecture with a clear separation between the presentation layer (React SPA), the application layer (Node.js REST API), and the data layer (PostgreSQL). All communication between client and server uses HTTPS with JSON payloads.

Architecture Components:

Component	Description
React SPA	Single-page application served from CDN or static hosting. Communicates with backend via REST API calls using Axios.
Node.js API Server	Express/Fastify-based REST API implementing the layered architecture. Handles all business logic, authentication, and data operations.
PostgreSQL Database	Persistent relational data store for all entities (users, teams, tasks, contributions, comments, resources).
JWT Auth Layer	Stateless authentication middleware that validates Bearer tokens on every protected request.
Background Jobs	Cron-based scheduler (node-cron) for deadline reminder notifications and contribution recalculations.

5.2 UI vs. UX Priority Decision

Taskora prioritizes User Experience (UX) over visual aesthetics (UI). This is a deliberate and justified decision based on the following reasoning:

The target users — student teams and startup members — are time-constrained and task-focused. They interact with Taskora to accomplish specific goals: checking their tasks, updating status, and reviewing their team's progress. Friction in these workflows directly translates to reduced adoption and abandoned sessions. A visually spectacular interface that is confusing to navigate fails its users.

Therefore, the design philosophy prioritizes:

- Clarity: Task status, deadlines, and assignments visible at a glance without requiring mental parsing.
- Efficiency: Critical actions (update status, add comment, mark complete) accessible within two clicks.
- Consistency: Predictable interaction patterns across all views to minimize learning curve.
- Responsiveness: Sub-200ms perceived response times through optimistic UI updates and proper loading states.
- Accessibility: WCAG 2.1 AA compliance for inclusive design.

UI quality is still important — Taskora will use a clean, professional design system (Tailwind CSS with a consistent color palette and typography scale) — but UI decisions are always subordinate to UX outcomes. If a visually appealing element creates confusion, it is redesigned or removed.

6. Backend Layered Architecture

The backend is structured as a strict five-layer architecture. Each layer has a single, well-defined responsibility and may only communicate with the layer directly adjacent to it. This constraint enforces separation of concerns and prevents the architectural degradation commonly seen in unstructured codebases.

Layer	Responsibility	Technologies
Router Layer	Route definition, middleware application, request routing	Express Router, auth middleware, rate limiter
Controller Layer	HTTP request/response handling, DTO validation, error forwarding	class-validator, class-transformer, Express handlers
Service Layer	Business logic, authorization checks, transaction coordination	Custom TypeScript services, transaction managers
Repository Layer	Database query abstraction, entity mapping	TypeORM / Prisma repositories, raw SQL for complex queries
Database Layer	Data persistence, schema enforcement, indexing	PostgreSQL 14+, migrations via TypeORM/Prisma Migrate

6.1 Router Layer

The Router Layer is the entry point for all HTTP requests. It is responsible for:

- Defining URL routes and mapping them to controller methods
- Applying middleware in the correct order: rate limiting, request logging, authentication, and role authorization
- Grouping routes by domain (e.g., /api/v1/tasks, /api/v1/teams, /api/v1/auth)
- No business logic resides in this layer — it is purely a routing and middleware orchestration layer

Example route definition:

```
router.patch('/tasks/:id/status', authenticate, authorize(['leader', 'member']),
taskController.updateStatus);
```

6.2 Controller Layer

The Controller Layer handles the HTTP contract. It is responsible for:

- Parsing and validating incoming request bodies using DTO classes decorated with class-validator
- Extracting path parameters, query strings, and authenticated user context
- Calling the appropriate service method with validated data

- Returning appropriate HTTP status codes (200, 201, 204, 400, 401, 403, 404, 409, 500) with structured JSON responses
- Forwarding errors to the centralized error handling middleware using next(error)

Controllers do not contain business logic. They are thin adapters between HTTP and the service layer. If validation fails, a 400 Bad Request is returned immediately without invoking the service.

6.3 Service Layer

The Service Layer is the heart of the application. It contains all business logic and orchestrates the following:

- Business rule enforcement (e.g., a member cannot verify their own task)
- Role-based authorization checks beyond simple RBAC (contextual permissions)
- Transaction management — wrapping multiple repository calls in database transactions to ensure atomicity
- Contribution score calculation and updates when task statuses change
- Cross-entity operations (e.g., creating a task also creates initial contribution records)

The Service Layer has no knowledge of HTTP. It receives plain TypeScript objects, performs operations, and returns domain objects or throws typed exceptions.

6.4 Repository Layer

The Repository Layer abstracts all database interactions. It is responsible for:

- Executing CRUD operations against the PostgreSQL database
- Mapping database rows to TypeScript entity objects
- Encapsulating complex queries (JOINs, aggregations, pagination) behind clean method signatures
- Handling database-specific errors and converting them to application-level exceptions

Using a Repository pattern (rather than direct ORM queries in services) means that if the database technology changes, only the repository implementations need to be modified — the service layer remains untouched. It also enables easy mocking in unit tests.

6.5 Database Layer

The Database Layer is the PostgreSQL instance managed via migration files. Its responsibilities are:

- Schema definition and enforcement (tables, columns, types, constraints)
- Foreign key relationships and cascading behaviors
- Indexing for query performance optimization

- Data persistence with ACID transaction support

6.6 Data Flow Through Layers

The following illustrates the data flow for a typical request — updating a task's status:

5. Client sends PATCH /api/v1/tasks/42/status with body { status: 'completed' } and Authorization: Bearer <token>
6. Router applies authenticate middleware (validates JWT, attaches user to request), then routes to taskController.updateStatus
7. Controller parses and validates the request body using UpdateTaskStatusDTO. If invalid, returns 400. Calls taskService.updateStatus(42, 'completed', user)
8. Service retrieves the task via taskRepository.findById(42), verifies the user has permission, checks business rules, then calls taskRepository.updateStatus(42, 'completed') and contributionRepository.incrementScore(task.assigneeId) within a single transaction
9. Repository executes the SQL UPDATE and INSERT within the transaction, returns the updated entity
10. Service returns the updated task DTO to the controller
11. Controller sends HTTP 200 with the task DTO as JSON

7. Functional Requirements

7.1 Authentication & Authorization

ID	Priority	Requirement
FR-AUTH-01	CRITICAL	The system shall allow users to register with email, name, and password. Passwords must be validated for minimum length (8 characters) and complexity.
FR-AUTH-02	CRITICAL	The system shall authenticate users via email and password, returning a signed JWT access token (expiration: 15 minutes) and a refresh token (expiration: 7 days).
FR-AUTH-03	CRITICAL	The system shall validate JWT tokens on all protected endpoints. Expired or invalid tokens must result in HTTP 401 Unauthorized.
FR-AUTH-04	HIGH	The system shall allow users to refresh their access token using a valid refresh token without re-entering credentials.
FR-AUTH-05	HIGH	The system shall enforce role-based access control. Access to resources must be checked against the user's role within the specific team workspace.
FR-AUTH-06	HIGH	The system shall allow users to log out, invalidating the refresh token on the server side.

7.2 Team & Workspace Management

ID	Priority	Requirement
FR-TEAM-01	CRITICAL	The system shall allow authenticated users to create a new team workspace with a name and description. The creator is automatically assigned the Team Leader role.
FR-TEAM-02	HIGH	Team Leaders shall be able to invite members to the workspace via email address. An invitation record is created and the invited user is notified.
FR-TEAM-03	HIGH	Team Leaders shall be able to remove members from the workspace. Removed members lose access immediately.
FR-TEAM-04	HIGH	Team Leaders shall be able to update member roles within the workspace (e.g., promote a Member to Co-Leader).
FR-TEAM-05	MEDIUM	The system shall display a roster of all team members with their roles and contribution scores on the workspace dashboard.

7.3 Task Management

ID	Priority	Requirement

FR-TASK-01	CRITICAL	Team Leaders shall be able to create tasks with: title, description, assignee (single member), priority (Low/Medium/High/Critical), deadline, and initial status (To-Do).
FR-TASK-02	CRITICAL	Team Members shall be able to update the status of tasks assigned to them through the workflow: To-Do -> In Progress -> Under Review.
FR-TASK-03	CRITICAL	Team Leaders shall be able to verify tasks in 'Under Review' status and transition them to 'Completed'. Verification triggers contribution score update.
FR-TASK-04	HIGH	The system shall display tasks in a filterable, sortable list with columns for title, assignee, priority, deadline, and status.
FR-TASK-05	HIGH	Team Leaders shall be able to edit task details (except assignee once In Progress) and delete tasks that are in To-Do status.
FR-TASK-06	MEDIUM	The system shall support task pagination (default 20 tasks per page) with cursor-based or offset-based pagination on the API.
FR-TASK-07	MEDIUM	Team Members shall be able to add comments to any task within their workspace. Comments are displayed in chronological order.

7.4 Contribution Tracking

ID	Priority	Requirement
FR-CONT-01	CRITICAL	The system shall maintain a contribution record per user per workspace. The contribution score is computed based on: completed tasks (+10 pts), on-time completion bonus (+5 pts), verified tasks (+3 pts).
FR-CONT-02	HIGH	The system shall display a ranked contribution leaderboard for each workspace, showing each member's score, number of completed tasks, and completion rate.
FR-CONT-03	HIGH	Contribution scores must be updated atomically with task status changes using database transactions to prevent data inconsistency.
FR-CONT-04	MEDIUM	Team Leaders shall be able to export contribution data as CSV for reporting purposes.

7.5 Calendar & Deadline Management

ID	Priority	Requirement
FR-CAL-01	HIGH	The system shall provide a calendar view that renders all task deadlines for the current workspace, filterable by assignee and priority.
FR-CAL-02	HIGH	The system shall send in-app notification reminders 24 hours before a task deadline to the assigned member.

FR-CAL-03	MEDIUM	The system shall visually highlight overdue tasks (past deadline and not completed) with a distinct color indicator.

7.6 Resource Sharing

ID	Priority	Requirement
FR-RES-01	MEDIUM	Team members shall be able to add shared folder shortcuts (name + URL) to the workspace resource panel.
FR-RES-02	MEDIUM	Team Leaders shall be able to delete any shared resource. Team Members may only delete resources they created.

8. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API responses for list endpoints must return within 300ms (p95) under normal load (100 concurrent users).
NFR-02	Performance	The frontend must achieve a Lighthouse Performance score of 80+ and First Contentful Paint under 2 seconds.
NFR-03	Security	All API endpoints must be protected by JWT authentication. Unauthenticated requests must return HTTP 401.
NFR-04	Security	Passwords must be hashed using bcrypt with a cost factor of minimum 10. Plain-text passwords must never be stored.
NFR-05	Security	All user inputs must be validated and sanitized. SQL injection must be prevented via parameterized queries / ORM.
NFR-06	Security	The API must implement rate limiting (100 requests/minute per IP) to prevent abuse and brute-force attacks.
NFR-07	Reliability	The system must target 99.5% uptime (excluding planned maintenance windows).
NFR-08	Reliability	All critical operations (task creation, status update, contribution change) must be wrapped in database transactions.
NFR-09	Scalability	The backend must be stateless (no server-side session storage) to support horizontal scaling behind a load balancer.
NFR-10	Scalability	The database schema must be indexing-optimized for queries on tasks.assignee_id, tasks.workspace_id, tasks.deadline, and tasks.status.
NFR-11	Maintainability	All backend code must be written in TypeScript with strict mode. No use of 'any' type without justification comment.
NFR-12	Maintainability	API errors must follow a consistent JSON structure: { success: false, error: { code, message, details } }.
NFR-13	Usability	The UI must be fully responsive and usable on screen widths from 375px (mobile) to 1920px (desktop).
NFR-14	Usability	The system must provide meaningful error messages to users — no raw stack traces or database error codes.
NFR-15	Compliance	User passwords and personal data must be handled in accordance with GDPR principles. Users must be able to delete their account and all associated data.

9. Database Design Overview

9.1 Entity Overview

The following core entities form the relational model for Taskora. All tables include created_at and updated_at timestamp columns managed by the ORM.

Entity	Key Attributes	Relationships
users	id, email (unique), name, password_hash, is_active	Has many workspace_members, task_assignees, comments
workspaces	id, name, description, created_by (FK users)	Has many workspace_members, tasks, resources
workspace_members	id, workspace_id (FK), user_id (FK), role (enum: leader/member/viewer)	Junction table: workspace <-> user with role
tasks	id, workspace_id (FK), title, description, assignee_id (FK users), priority (enum), status (enum), deadline	Belongs to workspace; assigned to user; has many comments
task_comments	id, task_id (FK), author_id (FK users), content, created_at	Belongs to task and author
contributions	id, workspace_id (FK), user_id (FK), score, tasks_completed, tasks_verified, on_time_count	One record per user per workspace; updated transactionally
resources	id, workspace_id (FK), created_by (FK users), name, url	Belongs to workspace
notifications	id, user_id (FK), type (enum), reference_id, is_read, created_at	Belongs to user; references tasks or workspaces
refresh_tokens	id, user_id (FK), token_hash, expires_at, revoked	Used for JWT refresh token rotation and revocation

9.2 Key Indexes

- tasks(workspace_id, status) — composite index for filtered task list queries
- tasks(assignee_id) — index for member-specific task queries
- tasks(deadline) WHERE status != 'completed' — partial index for deadline reminder jobs
- contributions(workspace_id, user_id) — unique constraint + index for leaderboard queries
- workspace_members(workspace_id, user_id) — unique constraint for membership integrity

9.3 Constraints and Data Integrity

- All foreign keys use ON DELETE CASCADE or ON DELETE SET NULL as appropriate to maintain referential integrity.
- Enum types (task status, priority, role) are enforced at the database level to prevent invalid values.
- The contributions table has a UNIQUE(workspace_id, user_id) constraint to prevent duplicate contribution records.
- The workspace_members table has a UNIQUE(workspace_id, user_id) constraint to prevent duplicate memberships.
- Database transactions are used whenever contribution scores are updated alongside task status changes.

10. Implementation Strategy

10.1 Development Approach

Taskora will be developed using an Agile-inspired iterative approach with bi-weekly sprint cycles. Development is organized into four phases:

Phase	Duration	Deliverables
Phase 1	Weeks 1-3	Project setup, database schema design, auth system (register, login, JWT), base architecture scaffolding
Phase 2	Weeks 4-7	Team workspace, member management, task CRUD, task status workflow, task comments
Phase 3	Weeks 8-10	Contribution tracking system, calendar view, deadline reminders, resource sharing, leaderboard
Phase 4	Weeks 11-13	UI polish, performance optimization, testing suite, deployment, documentation finalization

10.2 API Design

The API follows RESTful conventions with versioning via URL prefix (/api/v1/). All requests and responses use JSON. HTTP verbs are used semantically:

- GET — retrieve resources (never mutates state)
- POST — create new resources
- PUT — replace a resource entirely
- PATCH — partial update of a resource
- DELETE — remove a resource

Response envelope format:

```
Success: { success: true, data: {...}, meta: { pagination? } }
Error:   { success: false, error: { code, message, details? } }
```

10.3 Authentication & Authorization Flow

12. User submits credentials (POST /api/v1/auth/login). Backend validates password hash using bcrypt.compare().
13. On success, backend signs a JWT access token (15 min TTL) and a refresh token (7-day TTL). The refresh token hash is stored in the refresh_tokens table.
14. Client stores the access token in memory (not localStorage) and the refresh token in an httpOnly cookie to prevent XSS.

15. Every protected request includes Authorization: Bearer <access_token>. The authenticate middleware verifies the token using the JWT secret.
16. The authorize(roles) middleware checks the user's role in the specific workspace (from workspace_members table) for resource-level RBAC.
17. When the access token expires, the client calls POST /api/v1/auth/refresh with the cookie. The backend validates the token hash and issues a new token pair (refresh token rotation).

10.4 Contribution Tracking Logic

Contribution scores are calculated server-side and stored in the contributions table. They are updated in real time through the following rules:

- Task completed (status -> completed): +10 points to assignee's score, +1 to tasks_completed count
- On-time completion (completed before or on deadline): +5 bonus points, +1 to on_time_count
- Task verified by Team Leader: +3 points, +1 to tasks_verified count
- Task deleted after completion: -10 points (score cannot go below 0)

All score updates are executed within a database transaction alongside the task status update to ensure atomic consistency. The contribution score is never recalculated from scratch on read — it is maintained incrementally to avoid expensive aggregate queries on large datasets.

10.5 Validation & Error Handling

Input validation is implemented using the class-validator library with DTO classes in the Controller layer. Each DTO uses decorators such as @IsEmail(), @IsEnum(), @MinLength(), @IsDateString(), and @IsUUID() to enforce schema correctness before business logic is invoked.

Error handling follows a centralized approach via Express error-handling middleware:

- Validation errors (DTO) -> HTTP 400 Bad Request with field-level error details
- Authentication errors -> HTTP 401 Unauthorized
- Authorization errors -> HTTP 403 Forbidden
- Resource not found -> HTTP 404 Not Found
- Conflict (e.g., duplicate email) -> HTTP 409 Conflict
- Unexpected errors -> HTTP 500 Internal Server Error (no stack trace in production response)

All errors are logged with Winston (structured JSON logging) to stdout/file, including request ID, user ID, endpoint, and timestamp for traceability.

11. Deployment Strategy

11.1 Development Environment

- Frontend: Vite dev server (localhost:5173) with hot module replacement
- Backend: ts-node-dev with watch mode for auto-restart on file changes (localhost:3000)
- Database: PostgreSQL running in a Docker container for consistent local environments
- Environment variables: Managed via .env files (dotenv). .env.example committed to repository; actual .env files gitignored
- Linting: ESLint + Prettier enforced via pre-commit hooks (Husky + lint-staged)

11.2 Environment Configuration

The following environment variables must be configured:

```
DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, JWT_EXPIRES_IN, REFRESH_EXPIRES_IN,
PORT, NODE_ENV, FRONTEND_URL (CORS), BCRYPT_ROUNDS
```

11.3 Production Deployment

The production deployment uses a containerized approach with Docker:

- Frontend: React app built to static assets (npm run build) and served via Nginx in a Docker container, or deployed to Vercel/Netlify for zero-configuration CDN distribution.
- Backend: Node.js API containerized with a multi-stage Dockerfile (build stage + production stage). Deployed to a cloud VPS (DigitalOcean Droplet, Railway, or Render) or a container orchestration service.
- Database: Managed PostgreSQL service (Railway Postgres, Supabase, AWS RDS, or Render Postgres) for automated backups, monitoring, and failover.

11.4 Recommended Hosting Options

Component	Option A	Option B	Option C
Frontend	Vercel (free tier)	Netlify	Nginx on VPS
Backend	Railway	Render	DigitalOcean Droplet + Docker
Database	Railway Postgres	Supabase	AWS RDS (production)

11.5 CI/CD Pipeline

A basic CI/CD pipeline using GitHub Actions is recommended:

18. Push to feature branch: Runs linter (ESLint), TypeScript type checker (tsc --noEmit), and unit tests.
19. Pull Request to main: Runs full test suite including integration tests against a test database.
20. Merge to main: Triggers automatic deployment to the staging environment.
21. Manual release tag: Triggers deployment to production with a manual approval gate.

11.6 Scalability Considerations

- Horizontal scaling: The stateless JWT-based backend can be scaled by adding more server instances behind a load balancer (e.g., Nginx, Traefik, or cloud load balancer).
- Database read replicas: Read-heavy endpoints (task lists, leaderboard) can be routed to PostgreSQL read replicas to reduce primary database load.
- Caching: Redis can be introduced in a future version to cache workspace member lists and contribution leaderboards, reducing database queries on frequently accessed data.
- Background job queues: The notification scheduler can be upgraded to a queue-based system (BullMQ + Redis) for reliability under high load.

12. Use Cases

UC-01: User Registers and Joins a Team

Field	Detail
Actor	New User
Precondition	User has received a workspace invitation link or email.
Main Flow	1. User navigates to registration page. 2. Submits email, name, and password. 3. System validates input and creates account. 4. User clicks invitation link. 5. System adds user to workspace as Member.
Postcondition	User is authenticated and can view the workspace dashboard.
Exceptions	Email already registered: system returns 409 Conflict. Invitation expired: system returns 410 Gone.

UC-02: Team Leader Assigns and Tracks a Task

Field	Detail
Actor	Team Leader
Precondition	Leader is authenticated and workspace has at least one member.
Main Flow	1. Leader opens task creation form. 2. Fills in title, description, assignee, priority, and deadline. 3. Submits form. 4. System creates task with To-Do status. 5. Assignee receives notification. 6. Leader views task on project board.
Postcondition	Task is visible to all workspace members. Assignee sees it in their task list.
Exceptions	Deadline in the past: validation error. Assignee not in workspace: validation error.

UC-03: Member Completes a Task and Score Updates

Field	Detail
Actor	Team Member (assignee), Team Leader (verifier)
Precondition	Task exists with status In Progress, assigned to the member.
Main Flow	1. Member updates task status to Under Review. 2. Team Leader receives notification. 3. Leader reviews task and clicks Verify. 4. System transitions status to Completed. 5. Transaction atomically updates contribution score: +10 base, +5 if on time, +3 for verification.
Postcondition	Task is marked Completed. Member's contribution score is updated. Leaderboard reflects new standings.

Exceptions	Leader rejects task: status returns to In Progress with a comment.

UC-04: Deadline Reminder Notification

Field	Detail
Actor	System (automated), Team Member
Trigger	Cron job runs every hour. Queries tasks with deadline within 24 hours and status != completed.
Main Flow	1. Cron job executes. 2. System queries eligible tasks. 3. Creates notification records for each assignee. 4. Member sees notification badge in UI. 5. Member opens notification to navigate directly to the task.
Postcondition	Notification is marked as read after the member views it.

13. Future Enhancements

The following features are identified as valuable future enhancements for Taskora v2.0 and beyond. They are explicitly excluded from the initial release scope to maintain focus on core value delivery:

Feature	Target Version	Description
Discussion Forum	v2.0	Per-workspace threaded discussion forum for team announcements and general communication.
Google Calendar Sync	v2.0	Bidirectional sync of task deadlines with Google Calendar via OAuth 2.0 integration.
Real-time Collaboration	v2.0	WebSocket-based real-time updates for task status changes visible across all connected clients without page refresh.
Email Notifications	v2.0	Email delivery for deadline reminders and task assignments via SendGrid or AWS SES.
Analytics Dashboard	v2.1	Workspace-level analytics: task completion trends, average completion time, workload heatmaps by member.
Mobile Application	v3.0	React Native mobile app sharing API with the web application.
OAuth Login	v2.0	Sign in with Google and GitHub for reduced friction onboarding.
Advanced RBAC	v2.1	Custom role definitions with granular permission sets configurable per workspace.

Document Control

Version	Author	Date	Description
1.0	Taskora Team	2025	Initial release

Role	Name	Signature	Date
Tech Lead	Nicholas		
Product Owner	Yovela		
System Analyst	Duard		
Programmer	Raynald		

— End of Document —